

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture CFG - 3



Parse Trees

Consider the CFG for the properly balanced left & right parentheses

$S \rightarrow e, S \rightarrow SS, S \rightarrow (S)$

Consider the string $() ()$

This can be derived as

$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()(S) \Rightarrow ()()$

It can also be derived as

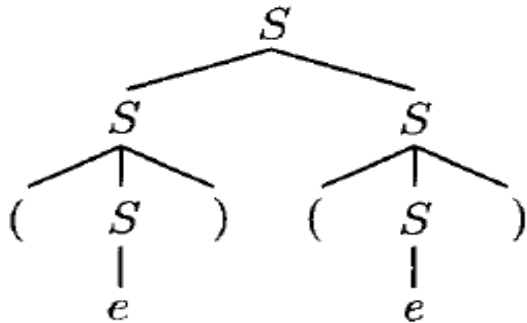
$S \Rightarrow SS \Rightarrow S(S) \Rightarrow S() \Rightarrow (S)() \Rightarrow ()()$

Observe:

- In these two derivations, *the rules applied at each step are same*
- *Except* the *difference is in the order in which the rules are applied.*

Parse Trees

We can view both derivations as



We call such a view as a *parse tree*.

The points are called *nodes*.

The topmost node is called the *root*.

The nodes along the bottom are called *leaves*.

By *concatenating* the labels of the *leaves* from left to right, we obtain the derived string, which is called the *yield* of the parse tree

Parse Trees

Formally, for an arbitrary CFG $G = \{V, \Sigma, R, S\}$, we define its *parse trees* and their *roots*, *leaves*, and *yields*, as follows:

1. $\circ a$

This is a *parse tree* for each $a \in \Sigma$. The single node of this parse tree is the *root* and a *leaf*. The *yield* of this parse tree is a .

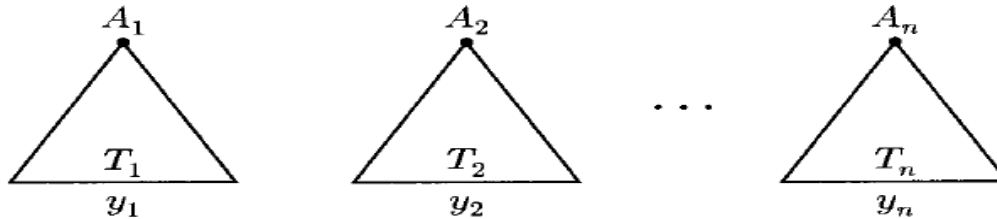
2. If $A \rightarrow e$ is a rule in R , then



is a *parse tree*; its *root* is the node labeled A , its sole *leaf* is the node labeled e , and its *yield* is e .

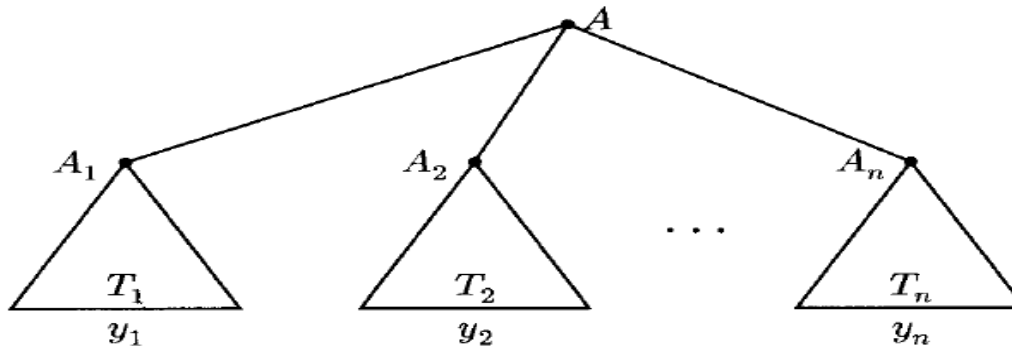
Parse Trees

3. If



are *parse trees*, where $n \geq 1$, with *roots* labeled

$A_1, A_2, \dots, A_n$ respectively and with *yields* $y_1, y_2, \dots, y_n$ and $A \rightarrow A_1 A_2 \dots A_n$ is a *rule* in R , then



is a *parse tree*. Its *root* is the new node labeled A , its *leaves* are the leaves of $T_1, T_2, \dots, T_n$ and the *yield* is

$y_1 y_2 \dots y_n$

Similar Derivations

Similar Derivations

$$D_1 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow (())S \Rightarrow (())(S) \Rightarrow (())()$$

$$D_2 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow ((S))(S) \Rightarrow (())(S) \Rightarrow (())()$$

Observe

The two derivations are *identical except* for *two consecutive steps*, during which the same two non-terminals are replaced by the same two strings *but in opposite orders in the two derivations*. The derivation in which the leftmost of the two non-terminals is replaced first is said to *precede* the other.

Parse Trees

Note

Parse trees are ways of representing derivations so that the *superficial differences* between derivations (because of order of applications of rules) are suppressed.

Parse Tree represent *equivalent derivations*.

Let $G = \{V, \Sigma, R, S\}$ be a CFG.

Let $D : x_1 \Rightarrow x_2 \Rightarrow \dots \Rightarrow x_n$

and $D' : x'_1 \Rightarrow x'_2 \Rightarrow \dots \Rightarrow x'_n$

be *two derivations* in G , where $x_i, x'_i \in V^*$ for $i=1,2,\dots,n$,
 $x_1, x'_1 \in V - \Sigma$ and $x_n, x'_n \in \Sigma^*$

Equivalent Derivations

We say that D *precedes* D' denoted $D < D'$ if $n > 2$ and there is an integer k , $1 < k < n$ such that

1. For all $i \neq k$ we have $x_i = x'_i$
2. $x_{k-1} = x'_{k-1} = uAvBw$ where $u, v, w \in V^*$ and $A, B \in V - \Sigma$
3. $x_k = uyvBw$, where $A \rightarrow y \in R$
4. $x'_k = uAvzw$, where $B \rightarrow z \in R$
5. $x_{k+1} = x'_{k+1} = uyvzw$

Example: Consider the three derivations D_1, D_2, D_3 then

$$D_1 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow (())S \Rightarrow (())(S) \Rightarrow (())()$$

$$D_2 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow ((S))(S) \Rightarrow (())(S) \Rightarrow (())()$$

$$D_3 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow ((S))(S) \Rightarrow ((S))() \Rightarrow (())()$$

Then $D_1 < D_2$
and $D_2 < D_3$
but D_1 *does not precede* D_3

We say D_1 and D_3 are *similar*

Equivalent Derivations

We say two derivations D and D' are *similar* if they belong in the reflexive, transitive closure of $<$

Leftmost Derivation

Repeatedly *replace* the *leftmost nonterminal* in the *current string* according to the rule suggested by the parse tree.

It is *not preceded* by any other derivation.

Rightmost Derivation

Repeatedly *replace* the *rightmost nonterminal* in the *current string* according to the rule suggested by the parse tree.

It *does not precede* any other derivation.

Ambiguous Grammars

CFG for Language of algebraic expressions involving the binary operators $+$ and $*$, left and right parentheses, and a single identifier id .

$E \rightarrow id \mid E + E \mid E * E \mid (E)$

Consider the string $id + id * id$

There are **two parse trees** for the string:

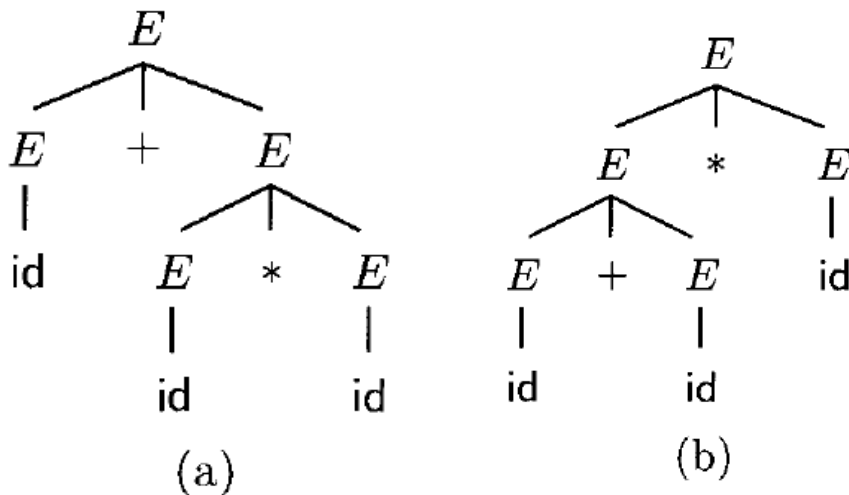


Figure (a) corresponds to the **natural** meaning of the expression as we know $*$ takes **precedence** over $+$

Such grammars with strings having two or more distinct parse trees are called **ambiguous**.

Unambiguous Grammars

Unambiguous grammar for algebraic expressions

Consider $id + id + id$

It should mean $(id + id) + id$ and not $id + (id + id)$

Think of *expression* as all but the last *term* plus the last *term*.

Idea

Replace $E \rightarrow E + E$ by $E \rightarrow E + T$, where variable T stands for *term*, an *expression* that cannot itself be expressed as a sum.

Expression could also be a single *term*.

We have $E \rightarrow E + T \mid T$

We now apply the same idea to the set of *terms*.

Terms can be *products*.

We introduce *factors* which are *terms* that can't be expressed as *products*

Unambiguous Grammars

The corresponding rules are:

$$T \rightarrow T * F \mid F$$

We have at this point:

Expressions are sum of one or more *terms*

Terms are *products* of one or more *factors*

This *hierarchy* incorporates *precedence* of * over + and they both *associate* to the left

Note that in our hierarchy *factors* are evaluated *first*.

So *parentheses* () should be considered as a *factor*

And inside the *parentheses* should be an *expression*.

$$F \rightarrow (E) \mid id$$

Unambiguous Grammars

Final Grammar is :

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

We need to show that this grammar is *equivalent* to the grammar of algebraic equations and is *unambiguous*.

We leave the details and be convinced with our intuition.

Consider the derivation of the string *id + id * id*

$$E \Rightarrow E + T \Rightarrow E + T * F \Rightarrow E + T * id \Rightarrow E + F * id \Rightarrow T + id * id$$

$$\Rightarrow F + id * id \Rightarrow id + id * id$$

This is the **only** derivation for the string
*id + id * id*

Ambiguity

Dangling Else

In the C programming language, an if-statement can be defined by these grammar rules:

$$S \rightarrow \text{if } (E) S \mid \\ \text{if } (E) S \text{ else } S \mid \\ OS$$

Consider `if (e1) if (e2) f() ; else g();`

In C the *else* should be associated with the *second if*, as in

$$\text{if } (e1) \{ \text{if } (e2) f(); \text{ else } g(); \}$$

There is nothing in these grammar rules to rule out the other interpretation:

$$\text{if } (e1) \{ \text{if } (e2) f(); \} \text{ else } g();$$

Unambiguous grammar

Equivalent grammar rules that allow only the correct interpretation

$$S \rightarrow S1 \mid S2$$
$$S1 \rightarrow \text{if } (E) S1 \text{ else } S1 \mid OS$$
$$S2 \rightarrow \text{if } (E) S \mid$$
$$\text{if } (E) S1 \text{ else } S2$$

Ambiguity: Computational Problems

Problem1: Given CFG G , find CFG G' such that $L(G) = L(G')$ and G' is unambiguous.

There is *no* algorithm that can solve the above problem!

Problem2: Given CFG G , determine if G is ambiguous.

The problem is *undecidable*.

Problem3: Is it the case that for every CFG G , there is a grammar G' such that $L(G) = L(G')$ and G' is unambiguous, even if G' cannot be constructed algorithmically?

No! There are context-free languages L such that every grammar for L is ambiguous.

Inherently ambiguous language

A context-free language L is said to be *inherently ambiguous* if *all* its grammars are *ambiguous*

Fortunately, programming languages are never *inherently ambiguous*.

Example of a language that can be proved *inherently ambiguous*.

$$L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$$

$$S \rightarrow AB \mid C$$

$$A \rightarrow aAb \mid ab \qquad B \rightarrow cBd \mid cd$$

$$C \rightarrow aCd \mid aDd \qquad D \rightarrow bDc \mid bc$$